50. Design Azure AI Foundry

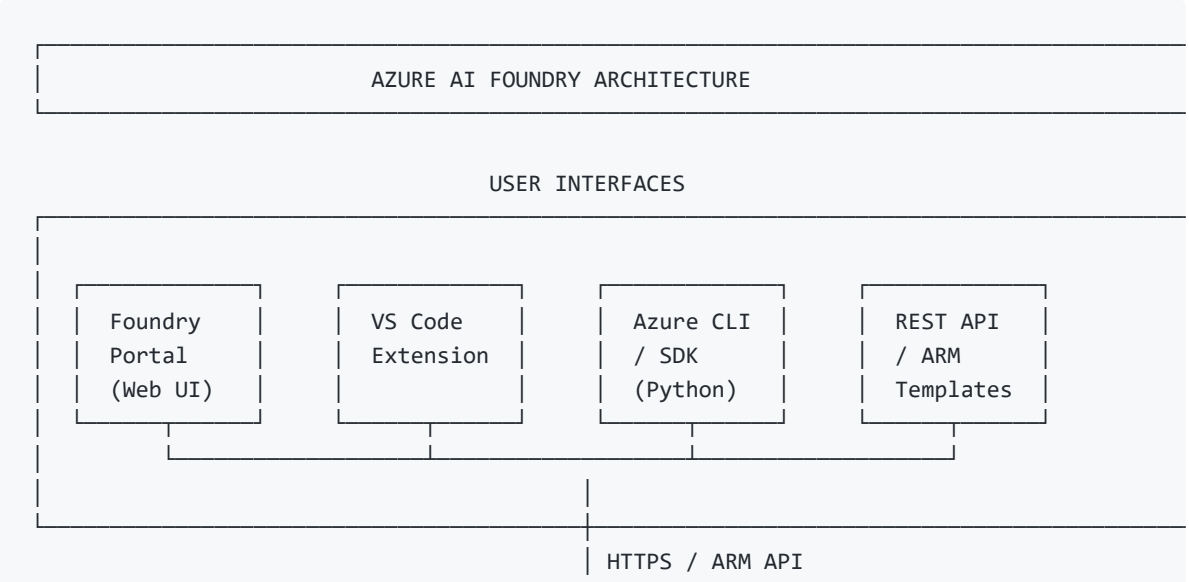
Problem Statement

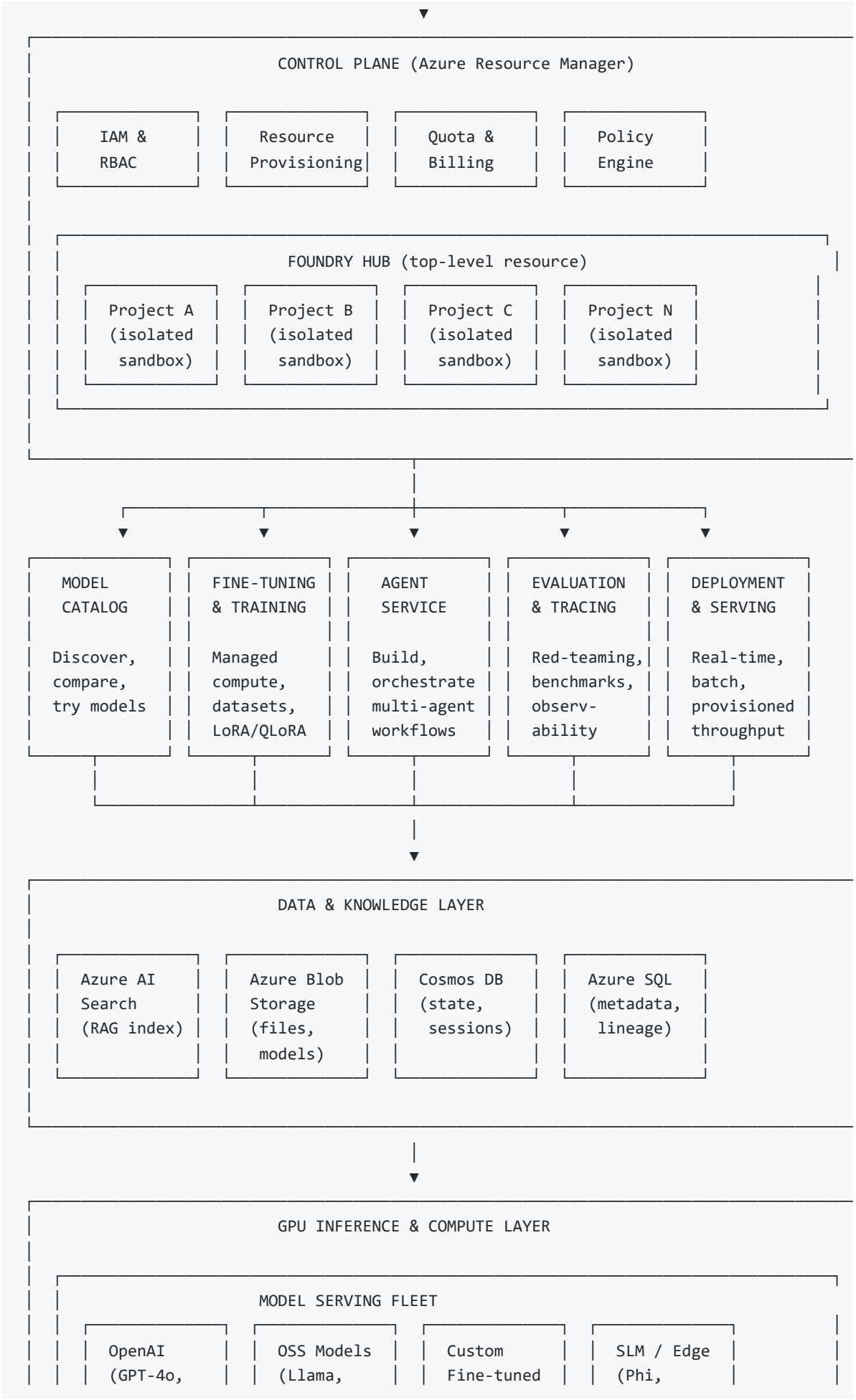
Design an enterprise-grade AI development platform — like Azure AI Foundry (formerly Azure AI Studio) — that enables teams to discover, customize, evaluate, deploy, and govern AI models and intelligent agents at scale. The platform must provide a unified experience across a massive model catalog, prompt engineering, fine-tuning, RAG pipelines, agent orchestration, and fleet-wide governance — all within a multi-tenant cloud environment.

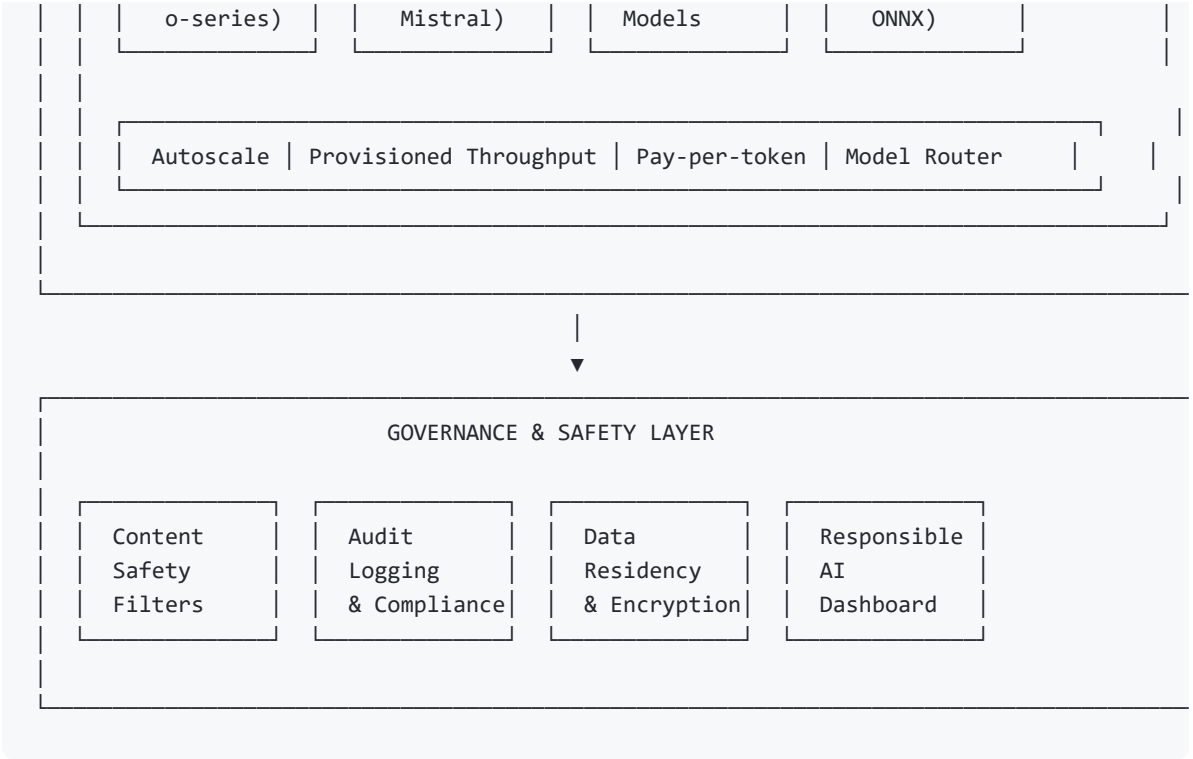
Clarifying Questions to Ask

1. **Scope**: Full MLOps lifecycle or just model serving and agent hosting?
2. **Multi-model**: How many model providers (OpenAI, Meta, Mistral, Hugging Face, custom)?
3. **Multi-tenant**: Per-project isolation or shared compute pools?
4. **Governance**: What compliance frameworks (GDPR, HIPAA, SOC 2, FedRAMP)?
5. **Deployment targets**: Cloud-only or also edge and on-premises?
6. **Agent complexity**: Single-agent tool-use or multi-agent orchestration?
7. **Scale**: How many concurrent model deployments and inference requests per tenant?
8. **Customization**: Fine-tuning on managed compute or bring-your-own training cluster?

High-Level Design




CONTROL PLANE (Azure Resource Manager)
IAM & RBAC
Resource Provisioning
Quota & Billing
Policy Engine
FOUNDRY HUB (top-level resource)
Project A (isolated sandbox)
Project B (isolated sandbox)
Project C (isolated sandbox)
Project N (isolated sandbox)
MODEL CATALOG
Discover, compare, try models
FINE-TUNING & TRAINING
Managed compute, datasets, LoRA/QLoRA
AGENT SERVICE
Build, orchestrate multi-agent workflows
EVALUATION & TRACING
Red-teaming, benchmarks, observ-ability
DEPLOYMENT & SERVING
Real-time, batch, provisioned throughput
DATA & KNOWLEDGE LAYER
Azure AI Search (RAG index)
Azure Blob Storage (files, models)
Cosmos DB (state, sessions)
Azure SQL (metadata, lineage)
GPU INFERENCE & COMPUTE LAYER
MODEL SERVING FLEET
OpenAI (GPT-4o,
OSS Models (Llama,
Custom Fine-tuned
SLM / Edge (Phi,



API Design

Project & Resource Management APIs

Method	Endpoint
`PUT`	`/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.AIFoundry/hubs/`
`PUT`	`.../hubs/{hub}/projects/{project}`
`GET`	`.../hubs/{hub}/projects`
`DELETE`	`.../hubs/{hub}/projects/{project}`

Model Catalog APIs

Method	Endpoint	Description

`GET`	`/v1/catalog/models`	List available models (filter by task, provider, license)
`GET`	`/v1/catalog/models/{modelId}`	Get model details (benchmarks, pricing, capabilities)
`POST`	`/v1/catalog/models/{modelId}/try`	Interactive playground — try a model with sample prompts
`POST`	`/v1/deployments`	Deploy a model (real-time, batch, or provisioned)

Inference APIs (Unified)

Method	Endpoint	Description
`POST`	`/v1/chat/completions`	Chat completion (OpenAI-compatible)
`POST`	`/v1/completions`	Text completion
`POST`	`/v1/embeddings`	Generate embeddings
`POST`	`/v1/images/generations`	Image generation
`POST`	`/v1/audio/transcriptions`	Speech-to-text

Agent APIs

Method	Endpoint	Description
`POST`	`/v1/agents`	Create an agent (with tools, instructions, model)
`POST`	`/v1/agents/{agentId}/threads`	Create a conversation thread
`POST`	`/v1/agents/{agentId}/threads/{threadId}/runs`	Run agent on a thread
`GET`	`/v1/agents/{agentId}/threads/{threadId}/runs/{runId}`	Get run status and output

Fine-Tuning APIs

Method	Endpoint	Description
`POST`	`/v1/fine-tuning/jobs`	Start a fine-tuning job
`GET`	`/v1/fine-tuning/jobs/{jobId}`	Get job status
`POST`	`/v1/fine-tuning/jobs/{jobId}/cancel`	Cancel a running job
`GET`	`/v1/fine-tuning/jobs/{jobId}/checkpoints`	List training checkpoints

Evaluation APIs

Method	Endpoint	Description
`POST`	`/v1/evaluations`	Run evaluation (groundedness, relevance, coherence)
`GET`	`/v1/evaluations/{evalId}/results`	Get evaluation results and metrics
`POST`	`/v1/evaluations/red-team`	Run automated red-team attack simulation

Model Deployment Request

```
POST /v1/deployments
{
  "name": "gpt-4o-production",
  "model": {
    "source": "catalog",
    "id": "openai/gpt-4o",
    "version": "2025-08-06"
  },
  "sku": {
    "name": "ProvisionedManaged",
    "capacity": 100
  },
  "scaling": {
    "type": "autoscale",
    "min_replicas": 2,
    "max_replicas": 10,
    "target_utilization": 0.7
  },
  "content_filter": {
    "policy": "default-v2",
    "custom_blocklists": ["company-terms"]
  },
  "networking": {
    "type": "private_endpoint",
    "vnet_id": "/subscriptions/.../virtualNetworks/ai-vnet"
  },
```

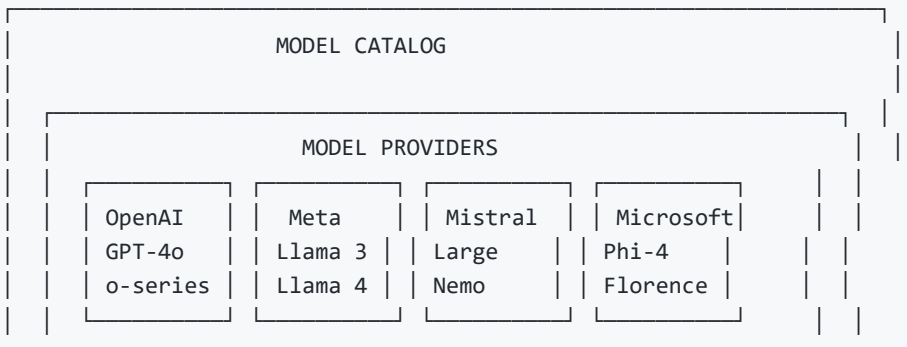
```
  "region": "eastus2"
}
```

Agent Creation Request

```
POST /v1/agents
{
  "name": "customer-support-agent",
  "model": "gpt-4o-production",
  "instructions": "You are a helpful customer support agent for Contoso...",
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "search_knowledge_base",
        "description": "Search internal support articles",
        "parameters": { "type": "object", "properties": { "query": { "type": "string
      }
    },
    {
      "type": "azure_ai_search",
      "config": {
        "index_name": "support-articles",
        "endpoint": "https://contoso-search.search.windows.net"
      }
    },
    { "type": "code_interpreter" }
  ],
  "temperature": 0.3,
  "max_tokens": 4096
}
```

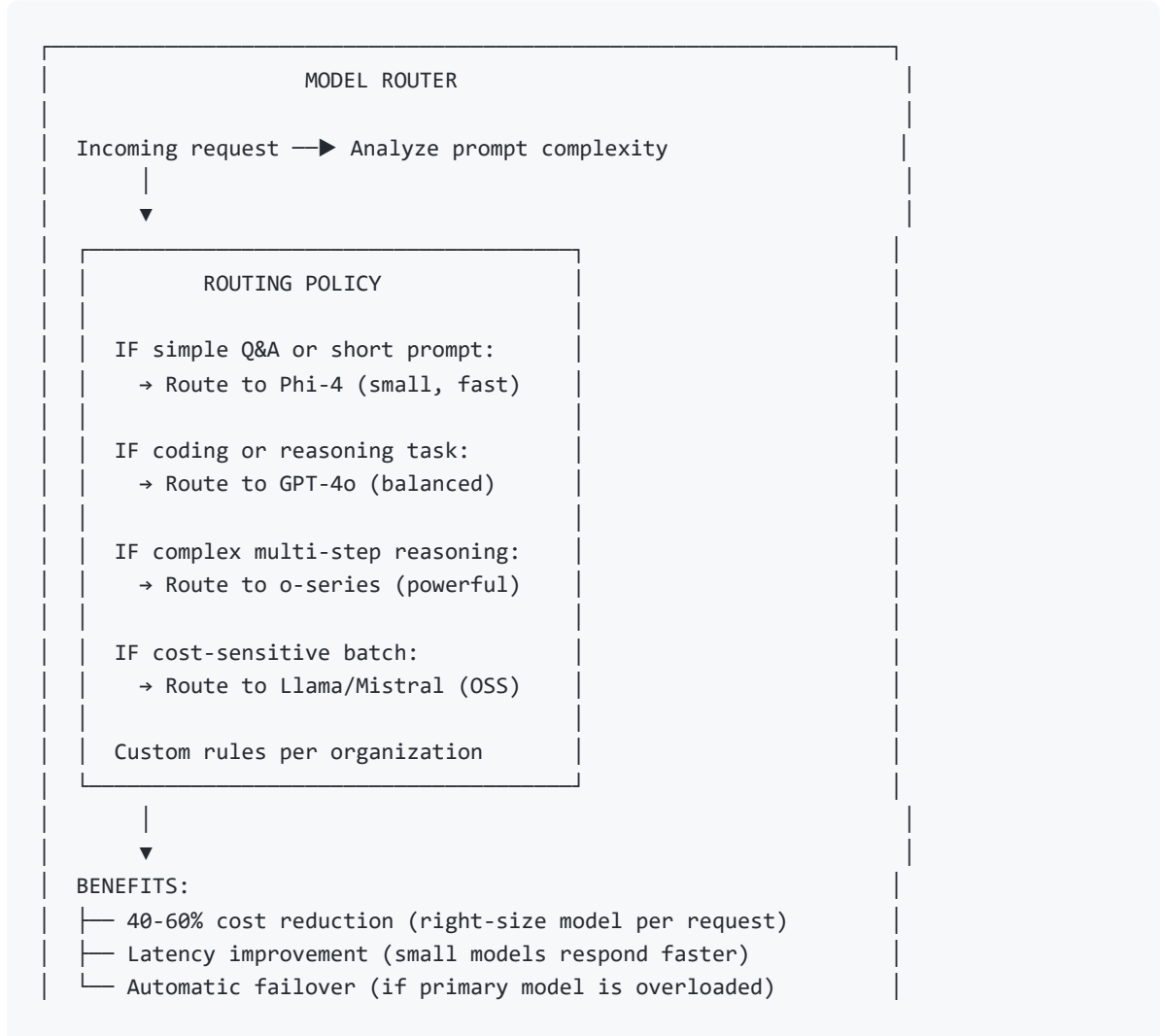
Model Catalog & Routing

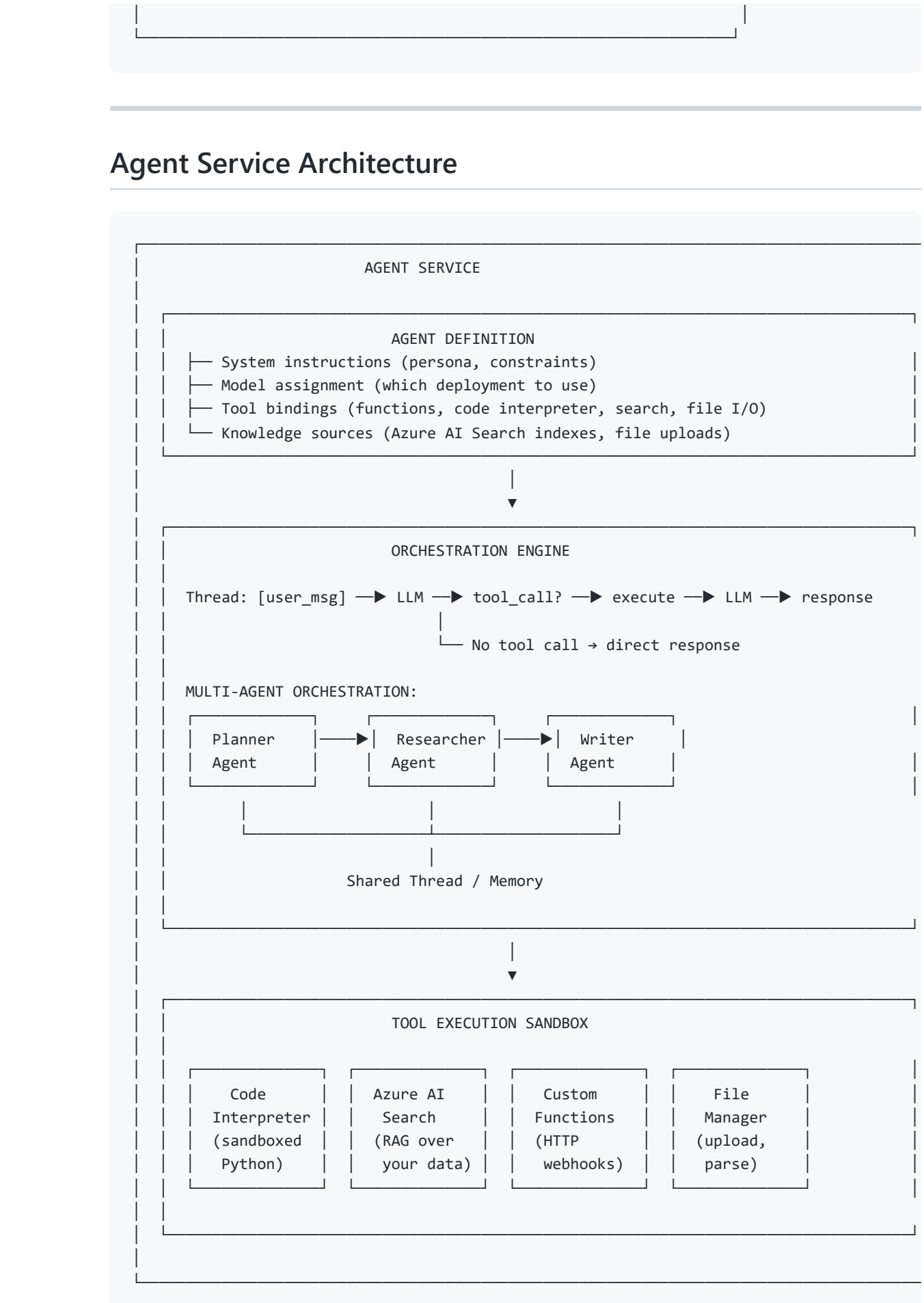
Model Catalog Architecture





Intelligent Model Router





Agent Service Architecture

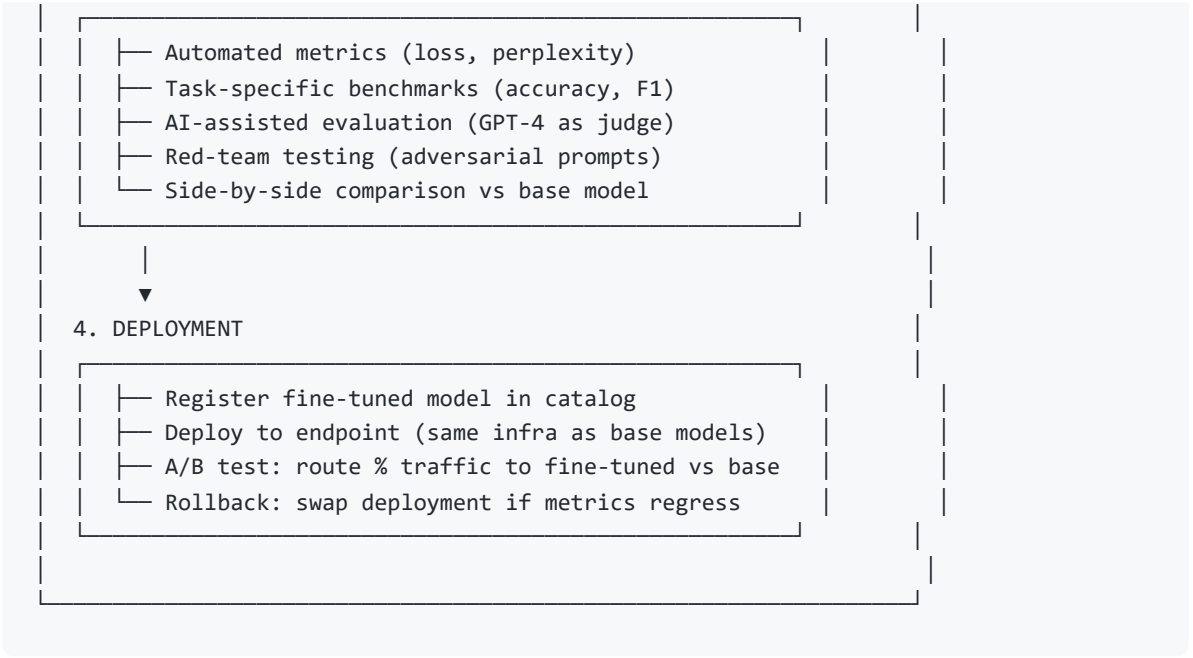
Agent Thread State Management

```
Thread (Cosmos DB document):
{
  "thread_id": "thread_abc123",
  "agent_id": "agent_support",
  "messages": [
    { "role": "user", "content": "What's my order status?" },
    { "role": "assistant", "content": null,
      "tool_calls": [{ "function": "lookup_order", "args": {"email": "..."} }] },
    { "role": "tool", "content": "Order #12345: Shipped, ETA Feb 25" },
    { "role": "assistant", "content": "Your order #12345 has shipped..." }
  ],
  "metadata": { "customer_id": "cust_789", "created_at": "..." },
  "ttl": 86400
}

State management strategy:
├── Short-term: Full message history in thread (Cosmos DB)
├── Medium-term: Summarize older messages to stay within context window
└── Long-term: Archive to blob storage, index key facts in AI Search
```

Fine-Tuning Pipeline

```
┌──────────────────────────────────────────────────────────────────┐
│                    FINE-TUNING PIPELINE                          │
│                                                                  │
│  1. DATA PREPARATION                                             │
│  ┌──────────────────────────────────────────────────┐            │
│  │  Upload training data (JSONL format)             │            │
│  │  ├── Validate schema (messages array per sample) │            │
│  │  ├── Check for PII / sensitive content           │            │
│  │  ├── Split: 80% train / 10% validation / 10% test│            │
│  │  └── Estimate token count → compute cost         │            │
│  └──────────────────────────────────────────────────┘            │
│       │                                                          │
│       ▼                                                          │
│  2. TRAINING (Managed Compute)                                   │
│  ┌──────────────────────────────────────────────────┐            │
│  │  ├── Method: LoRA (default) or QLoRA or Full     │            │
│  │  │   └── LoRA: Train only adapter weights (~1-5% of │        │
│  │  │       parameters), dramatically lower GPU cost   │        │
│  │  ├── Hyperparameters: epochs, learning_rate, batch  │        │
│  │  ├── Compute: Auto-provisioned GPU cluster          │        │
│  │  │   └── ND A100/H100 nodes, scaled per model size  │        │
│  │  └── Checkpointing: Save every N steps              │        │
│  └──────────────────────────────────────────────────┘            │
│       │                                                          │
│       ▼                                                          │
│  3. EVALUATION                                                   │
```



Evaluation & Observability



```
│  │  ├── Span: API Gateway (5ms)                                    │    │
│  │  ├── Span: Content Safety Input Filter (12ms)                    │    │
│  │  ├── Span: RAG Retrieval (45ms)                                  │    │
│  │  │   └── AI Search query: "order return policy"                  │    │
│  │  │   └── Top-3 results, scores: [0.92, 0.87, 0.81]               │    │
│  │  ├── Span: Prompt Assembly (8ms)                                 │    │
│  │  │   └── Total tokens: 2,340 (prompt) + 512 (completion)         │    │
│  │  ├── Span: Model Inference (320ms)                               │    │
│  │  │   └── Model: gpt-4o, deployment: prod-east                    │    │
│  │  ├── Span: Content Safety Output Filter (10ms)                   │    │
│  │  └── Span: Response Streaming (200ms)                            │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  DASHBOARDS:                                                             │
│  ├── Request volume, latency p50/p95/p99                                 │
│  ├── Token consumption by model, project, user                           │
│  ├── Error rates and content filter trigger rates                        │
│  ├── Evaluation metric trends over time                                  │
│  └── Cost breakdown by project and deployment                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Hub & Project Isolation Model

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MULTI-TENANT ISOLATION                                │
│                                                                          │
│  AZURE SUBSCRIPTION                                                      │
│  └── RESOURCE GROUP                                                      │
│      └── FOUNDRY HUB (org-level)                                         │
│          ├── Shared resources:                                           │
│          │   ├── Azure AI Search instance                                │
│          │   ├── Azure Key Vault (secrets, API keys)                     │
│          │   ├── Storage account (models, datasets)                      │
│          │   └── Managed network (VNet, private endpoints)               │
│          │                                                               │
│          ├── PROJECT A (team sandbox)                                    │
│          │   ├── Own model deployments                                   │
│          │   ├── Own connections (to data sources)                       │
│          │   ├── Own agents and evaluation runs                          │
│          │   ├── RBAC: Project-scoped roles                              │
│          │   └── Cost tracking: Tagged per project                       │
│          │                                                               │
│          ├── PROJECT B                                                   │
│          │   └── ... (fully isolated from Project A)                     │
│          │                                                               │
│          └── PROJECT N                                                   │
│                                                                          │
│  ISOLATION GUARANTEES:                                                   │
│  ├── Network: Private endpoints, no public internet                      │
│  ├── Data: Each project's data in separate containers                    │
```

```
│  ├── Compute: Dedicated or shared GPU pools (configurable)       │
│  ├── RBAC: Azure AD roles scoped to hub or project               │
│  ├── Secrets: Key Vault access policies per project              │
│  └── Billing: Cost attribution per project via tags              │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Technology	Purpose
Control Plane	Azure Resource Manager (ARM)	Hub/project provisioning, RBAC, policy enforcement
Model Catalog	Azure SQL + Blob Storage	Model registry, metadata, benchmarks, versioning
Model Serving	Azure ML Managed Endpoints / MaaS	Real-time inference with autoscaling
Model Router	Custom routing service	Intelligent model selection per request
Agent Service	Cosmos DB + AKS	Agent orchestration, thread management, tool execution
Code Interpreter	Hyper-V sandboxed containers	Secure Python execution for agents
RAG / Search	Azure AI Search	Vector + keyword hybrid search over user data
Fine-Tuning	Azure ML Compute (GPU VMs)	LoRA/QLoRA training with managed infra
Evaluation	Azure ML Pipelines	Automated quality, safety, and red-team evaluation
Tracing	Azure Monitor + OpenTelemetry	End-to-end request tracing, latency, token usage
Content Safety	Azure AI Content Safety	Input/output filtering for harmful content
Key Management	Azure Key Vault	API keys, connection strings, certificates
Networking	Azure VNet + Private Link	Network isolation, no data exfiltration

CI/CD	GitHub Actions + Azure DevOps	Prompt/model deployment pipelines

Scalability Considerations

1. Inference Scaling

```
Challenge: Bursty, unpredictable inference traffic across thousands of tenants

Solutions:
├── Provisioned Throughput Units (PTU)
│   └── Pre-purchase fixed capacity for predictable workloads
│       (e.g., 100 PTU = ~600 requests/min for GPT-4o)
├── Pay-per-token (serverless)
│   └── Scale to zero, pay only for usage (higher per-token cost)
├── Autoscaling managed endpoints
│   └── Scale replicas 1→N based on request queue depth
├── Model Router
│   └── Shift overflow traffic to cheaper/smaller models
└── Global deployment
    └── Deploy same model across regions, route to nearest
```

2. Multi-Region Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    GLOBAL DEPLOYMENT                     │
│                                                          │
│   US East ◄──────────────────────────────► US West       │
│   Hub + GPU cluster     Azure Front Door    Hub + GPU cluster  │
│      │                                          │        │
│      └──────────── EU West ◄─── APAC ──────────┘        │
│                   Hub + GPU          Hub + GPU           │
│                                                          │
│   Data residency: Models and data stay in declared region      │
│   Failover: Cross-region for availability, not data migration  │
│   Catalog: Global read replicas, regional write primary        │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

3. Cost Management

```
Per-project cost tracking:
├── Token consumption metered per deployment per project
├── Fine-tuning compute hours billed per job
├── Storage (datasets, model artifacts) billed per GB
```

```
├── Search index queries billed per transaction
└── Quota system prevents runaway spend

Cost optimization levers:
├── Model Router: Auto-select cheapest model that meets quality bar
├── Caching: Semantic cache for repeated queries (~10-15% savings)
├── Batch API: 50% discount for non-real-time workloads
├── Spot/low-priority compute for fine-tuning jobs
└── Scale-to-zero serverless for dev/test deployments
```

Trade-offs

Decision	Option A	Option B	Foundry's Choice
Model hosting	Single-provider (OpenAI only)	Multi-provider catalog	**Multi-provider** — choice, flexibility, cost optimization
Deployment model	Shared multi-tenant GPUs	Dedicated per-customer	**Both** — MaaS (shared) and Managed Compute (dedicated)
Agent state	Stateless (client manages)	Server-side threads	**Server-side threads** — simpler client, reliable state
Fine-tuning compute	Customer-managed GPU	Platform-managed	**Platform-managed** — lower friction, auto-provisioned
RAG approach	Built-in vector DB	Bring-your-own search	**Azure AI Search** — integrated, but supports external connectors
Content safety	Optional per deployment	Always-on	**Always-on by default** — can customize severity thresholds
Evaluation	Manual only	Automated + manual	**Automated + manual** — CI/CD integrated evaluation pipelines
Pricing	Flat subscription	Pay-per-token	**Pay-per-token + PTU** — flexibility for different workloads

---

Technologies Deep Dive

AI & Model Serving

Technology	Purpose	Why
Azure OpenAI Service	Hosted OpenAI models	Enterprise SLA, content safety, private networking
Azure ML Managed Endpoints	OSS model serving	vLLM-backed, autoscaling, GPU node pools
ONNX Runtime	Optimized inference	Cross-platform, quantization, edge deployment
Triton Inference Server	Multi-framework serving	Batching, model ensembles, GPU scheduling
LoRA / QLoRA	Parameter-efficient fine-tuning	Fine-tune large models on single GPU node

Platform Infrastructure

Technology	Purpose	Why
Azure Resource Manager	Control plane	Consistent API, RBAC, policy, ARM templates
Azure Kubernetes Service	Agent + API hosting	Scalable microservices, GPU node pools
Cosmos DB	Agent threads, session state	Global distribution, low-latency, TTL
Azure AI Search	RAG retrieval	Hybrid search (vector + BM25), semantic ranker
Azure Key Vault	Secrets management	API keys, certificates, managed identities
Azure Monitor + App Insights	Observability	Distributed tracing, metrics, alerts
Azure Event Hubs	Telemetry streaming	Usage events, audit logs, billing events

Developer Tooling

Technology	Purpose	Why

Foundry SDK (Python)	Programmatic access	Unified client for all Foundry APIs
Prompt Flow	Prompt orchestration	Visual DAG for chaining prompts, tools, evaluators
VS Code Extension	IDE integration	Inline prompt testing, deployment, debugging
GitHub Actions	CI/CD for AI apps	Automated evaluation + deployment pipelines

---

How to Explain It Well

30-Second Pitch

> "Azure AI Foundry is a unified platform for building AI applications and agents at enterprise scale. It provides a model catalog with thousands of models from OpenAI, Meta, Mistral, and others — deployed as managed endpoints with autoscaling. On top of model serving, it offers agent orchestration with tool-calling, RAG via Azure AI Search, fine-tuning with LoRA, automated evaluation including red-teaming, and fleet-wide governance. The architecture is built on a Hub-and-Project isolation model where each project gets its own sandbox with scoped RBAC, networking, and cost tracking — all managed through Azure Resource Manager."

Key Talking Points

1. **Model Catalog + Router**: Not just hosting one model — curating and intelligently routing across hundreds of models for cost and quality
2. **Hub/Project isolation**: Enterprise multi-tenancy with project-level sandboxing (RBAC, network, data, billing)
3. **Agent orchestration**: Server-side thread management with tool execution in sandboxed environments
4. **Evaluation-as-a-service**: Automated quality + safety metrics integrated into CI/CD, not an afterthought
5. **Responsible AI by default**: Content safety filters always-on, red-team testing built in, audit logging for compliance

Whiteboard Sequence

1. Start with Users → Control Plane → Hub/Projects hierarchy
2. Show the Model Catalog and deployment types (MaaS, Managed, Serverless)

3. Draw the Agent Service with tool execution and thread state
4. Add RAG pipeline (data → AI Search → prompt → model → response)
5. Overlay Governance layer (content safety, audit, RBAC)

Common Follow-up Questions

"How do you handle model versioning and rollback?"

- Each model deployment references a specific model version (e.g., `gpt-4o-2025-08-06`)
- **Blue/green deployments**: Deploy new version alongside old, shift traffic gradually
- **Evaluation gates**: Automated eval runs must pass before traffic shift
- **Instant rollback**: Swap deployment pointer back to previous version (< 30s)
- Fine-tuned models are registered with version IDs and linked to training data lineage

"How does the model router decide which model to use?"

- **Prompt classifier**: Lightweight ML model analyzes prompt complexity (simple QA vs. complex reasoning)
- **Cost policy**: Org-level rules like "prefer models under $X per 1K tokens"
- **Quality threshold**: Route to smallest model that meets the quality bar (measured via evaluations)
- **Fallback chain**: If primary model is overloaded or unavailable, cascade to alternatives
- **Custom rules**: Organizations define routing logic (e.g., "use GPT-4o for legal, Phi for internal FAQ")

"How do you ensure data privacy in a multi-tenant platform?"

- **Network isolation**: Private endpoints — model traffic never traverses public internet
- **Data residency**: Models and data stay in the declared Azure region
- **No cross-tenant training**: Customer data is never used to train shared models
- **Encryption**: At-rest (AES-256) and in-transit (TLS 1.3), customer-managed keys supported
- **RBAC + Managed Identity**: No shared API keys — Azure AD authentication throughout
- **Audit logging**: Every API call, model invocation, and data access logged to Azure Monitor

"How do you handle RAG at scale?"

- **Azure AI Search** provides hybrid retrieval (vector embeddings + BM25 keyword search)
- **Semantic ranker** re-ranks results using a cross-encoder model for higher relevance

- **Chunking strategies**: Configurable document chunking (fixed-size, sentence-level, semantic)
- **Index per project**: Each project has isolated search indexes (no cross-project data leakage)
- **Integrated ingestion**: Upload docs → automatic chunking → embedding → indexing pipeline
- **Grounding evaluation**: Automated checks that responses cite retrieved context accurately

"How does fine-tuning work without managing GPUs?"

- User uploads training data (JSONL) and selects base model
- Platform auto-provisions GPU compute (ND A100/H100 VMs) based on model size
- **LoRA** (default): Only trains small adapter layers (~1-5% of parameters) — faster, cheaper
- Training runs on managed AML compute with automatic checkpointing
- After training, fine-tuned model is registered and deployable like any catalog model
- GPU compute is released immediately after job completes — no idle cost